

3.3 转换目标与转换方式

ConversionTarget 的合法性动作接口

转换目标用于指定在转换过程中合法或不合法的方言或操作, 从而指定整个重写和 转换的应用范围。

操作和方言可以通过 ConversionTarget 的三类合法性动作接口标记为是否合法。

- 指定合法 (legal) 接口: addLegalDialect(), addLegalOp()
- 指定非法 (illegal) 接口: addIllegalDialect(), addIllegalOp()
- 指定动态 (dynamic) 接口: addDynamicallyLegalDialect(), addDynamicallyLegalOp()

```
struct ConvertNVGPUToNVVMPass
: public impl::ConvertNVGPUToNVVMBase<ConvertNVGPUToNVVMPass> {
...
void runOnOperation() override {
...
    LLVMConversionTarget target(getContext());
    target.addLegalDialect<::mlir::LLVM::LLVMDialect>();
    target.addLegalDialect<::mlir::arith::ArithDialect>();
    target.addLegalDialect<::mlir::memref::MemRefDialect>();
    target.addLegalDialect<::mlir::NVVM::NVVMDialect>();
    ...
    applyPartialConversion(..., target,...);
};
```

转换方式与合法性的关系

为了适配不同的使用场景, MLIR 在做操作转换时提供了部分转换(partial conversion)、完全转换(full conversion)和分析转换(analysis conversion)三种不同的转换方式, 对应的执行转换接口分别为 `applyPartialConversion()`、`applyFullConversion()` 和 `applyAnalysisConversion()`。

```
func.func @async_cp(...) {  
  ...  
  %0 = nvgpu.device_async_copy %src[%i, %i], %dst[%i, %i, %i], 4 : memref<128x128xf32> to memref<3x16x128xf32, 3>  
  %1 = nvgpu.device_async_create_group %0  
  nvgpu.device_async_wait %1 { numGroups = 1 : i32 }  
  ...  
}
```

情况一



没有明确指定非法
没有添加重写模式

情况二



明确指定非法
没有添加重写模式

```
...  
nvvm.cp.async.shared.global %11, %18, 16  
...  
%21 = nvgpu.device_async_create_group %20  
nvvm.cp.async.wait.group 1  
...
```

报错:

failed to legalize operation 'nvgpu.device_async_create_group'
that was explicitly marked illegal

操作合法化过程 - 1

LogicalResult

```
mlir::applyPartialConversion(...) {  
    OperationConverter opConverter(target, patterns, ...Partial,...);  
    return opConverter.convertOperations(ops);  
}
```

LogicalResult

```
OperationConverter::convertOperations(ArrayRef<Operation *> ops, ...) {  
    ...  
    SmallVector<Operation *> toConvert;  
    ...  
    for (auto *op : toConvert)  
        if (failed(convert(rewriter, op)))  
            return rewriterImpl.discardRewrites(), failure();  
    ...  
}
```

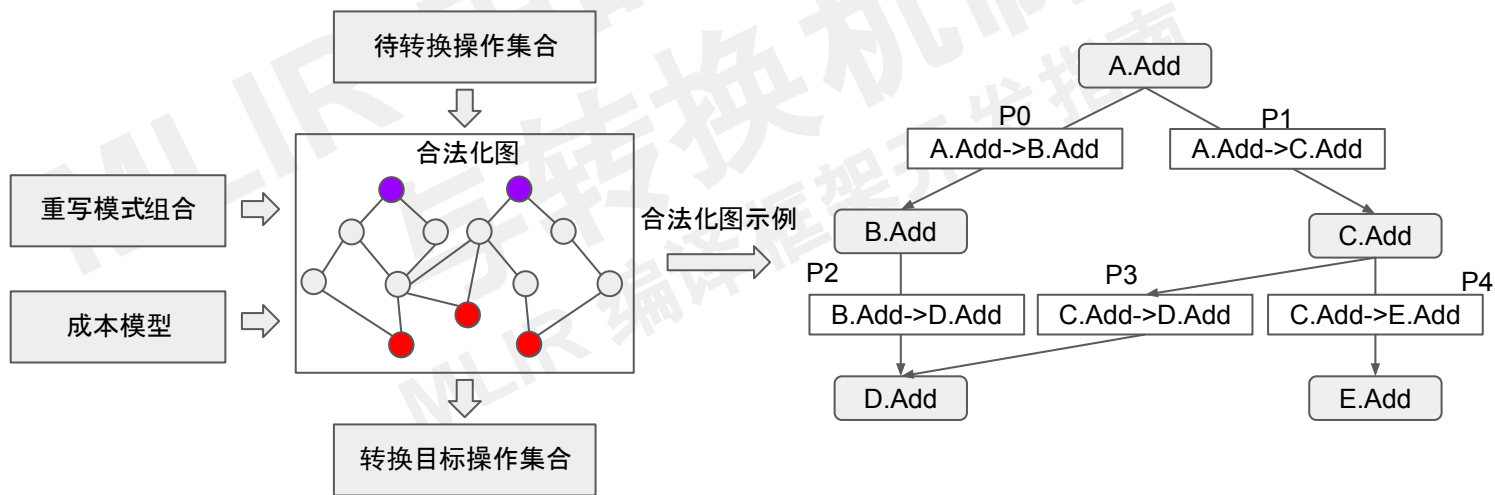
LogicalResult

```
OperationConverter::convert(...) {  
    if (failed(opLegalizer.legalize(op, rewriter))) {  
        if (mode == OpConversionMode::Full)  
            return op->emitError() << "failed to legalize operation '...'";  
        if (mode == OpConversionMode::Partial) {  
            if (opLegalizer.isIllegal(op))  
                return op->emitError() << "failed to legalize operation ...";  
            if (trackedOps)  
                trackedOps->insert(op);  
        }  
        else if (mode == OpConversionMode::Analysis) {  
            trackedOps->insert(op);  
        }  
        return success();  
    }  
}
```

操作合法化过程中的合法化图

操作合法化主要涉及待转换操作(源操作)、转换目标操作(目标操作)和二者之间的重写模式。

合法化图定义了在给定的重写模式集合和转换目标条件下,待转换操作集合与转换目标操作集合之间的合法化路径和约束条件。



操作合法化过程 - 2

LogicalResult

```
OperationLegalizer::legalize(Operation *op, ... &rewriter) {  
    ...  
    if (succeeded(legalizeWithFold(op, rewriter))) { return success(); }  
    if (succeeded(legalizeWithPattern(op, rewriter))) { return success(); }  
    ...  
    return failure();  
}
```

LogicalResult

```
OperationLegalizer::legalizeWithPattern(Operation *op, ... &rewriter) {  
    ...  
    auto canApply = [&](const Pattern &pattern) { ... };  
    auto onFailure = [&](const Pattern &pattern) { ... };  
    auto onSuccess = [&](const Pattern &pattern) { ... };  
    return applicator.matchAndRewrite(op, rewriter,  
                                       canApply, onFailure, onSuccess);  
}
```

普通模式列表
patterns

操作名称0	→	重写模式(0,0)	...	重写模式(0,m)
操作名称1	→	重写模式(1,0)	...	重写模式(1,m)
.....	→		...	
操作名称n	→	重写模式(n,0)	...	重写模式(n,m)

通配模式列表
anyOpPatterns

重写模式(0)	...	重写模式(k)
---------	-----	---------

重写模式的 matchAndRewrite() 函数调用栈

